

Sym6-Tuto-01

Last up dated 08.2025

// Creating a New Symfony Project

<https://symfony.com/doc/6.4/setup.html>

Sym6ShopH Project

cd C:\HONG\PHPtest25\SYMFONY6

// using the Symfony binary

// using Composer

composer create-project symfony/skeleton:"6.4.*" Sym6ShopH

cd Sym6ShopH

composer require webapp

// start project

the project has been created, start it using the following commands:

cd C:\HONG\PHPtest25\SYMFONY6\Sym6ShopH

symfony server:start or

php -S localhost:8000 -t public/

You can now access the app at <http://localhost:8000>

// the .env file

// the database configuration

Now update the database-relevant values inside the .env file

DATABASE_URL="mysql://root:Vuanh--66@127.0.0.1:3306/sym6shoph"

// install package

// install the Doctrine orm package

composer require symfony/orm-pack

// install and enable Maker bundle in your application

composer require --dev symfony/maker-bundle

// data

// create the database

Now let's create the database by running the following command: // => sym6shoph DB

php bin/console doctrine:database:create

// create an entity

You will need an entity class to represent your data object. Let's create it by running the following command:

php bin/console make:entity

You've added all the defined fields into Entity attributes.

// migration

you can now create your migration to create the table in the database. Run the following:

php bin/console make:migration

php bin/console doctrine:migrations:migrate

// Migrations & Adding more Fields

<https://symfony.com/doc/6.4/doctrine.html#migrations-adding-more-fields>

php bin/console make:entity

// Product

// generating-controllers**// option 1**

```
// php bin/console make:controller [EntityName]Controller
php bin/console make:controller ProductController
```

// option 2. If you want to generate an entire CRUD from a Doctrine entity, use:

```
// php bin/console make:crud Entity-Name
php bin/console make:crud Product
```

// result page for option 2<http://localhost:8000/product>**PART 2****// Using Doctrine DBAL**

An abstraction layer that sits on top of PDO

The DBAL library allows you to write queries in-dependently of your ORM models

<https://symfony.com/doc/current/doctrine/dbal.html>

use Doctrine\DBAL\Connection;

class UserController extends AbstractController {

```
    public function index(Connection $connection): Response {
```

```
        $users = $connection->fetchAllAssociative('SELECT * FROM users');
```

```
        ...
```

```
    }
```

// Forms**// Creating Form Classes**<https://symfony.com/doc/current/forms.html#creating-form-classes><https://symfony.com/doc/current/forms.html#form-type-options>

```
namespace App\Form\Type;
```

```
use Symfony\Component\Form\AbstractType;
```

```
class TaskType extends AbstractType
```

// Creating Forms in Controllers<https://symfony.com/doc/current/forms.html#creating-forms-in-controllers>

```
class TaskController extends AbstractController
```

```
    public function new(): Response {
```

```
        $task = new Task();
```

```
        $form = $this->createForm(TaskType::class, $task);
```

// processing-forms<https://symfony.com/doc/current/forms.html#processing-forms><https://symfony.com/doc/current/forms.html#validating-forms><https://symfony.com/doc/current/forms.html#rendering-forms>**// How to Upload Files**https://symfony.com/doc/6.4/controller/upload_file.html<https://symfony.com/doc/current/forms.html#form-type-options>

// Services

// Creating/Configuring Services

https://symfony.com/doc/6.4/service_container.html#creating-configuring-services-in-the-container

https://symfony.com/doc/6.4/service_container.html#service-parameters

// Automatic Service Loading in services.yaml

config/services.yaml

services:

 # default configuration for services in *this* file

 _defaults:

 autowire: true # Automatically injects dependencies in your services.

 autoconfigure: true # Automatically registers your services as commands, event subscribers, etc.

 # makes classes in src/ available to be used as services

 # this creates a service per class whose id is the fully-qualified class name

 App\:

 resource: '../src/'

 exclude:

 - '../src/DependencyInjection/'

 - '../src/Entity/'

 - '../src/Kernel.php'

Thanks to this configuration, you can automatically use any classes from the src/ directory as a service, without needing to manually configure it.

// Explicitly Configuring Services and Arguments

https://symfony.com/doc/6.4/service_container.html#explicitly-configuring-services-and-arguments

// Injecting Services/Config into a Service

https://symfony.com/doc/6.4/service_container.html#injecting-services-config-into-a-service

// Fetching and using Services

<https://symfony.com/doc/current/controller.html#fetching-services>

In your /controller, you can "ask" for /a service from the container by /type-hinting an argument with the service's class or interface name.

```
use Psr\Log\LoggerInterface;
```

```
class ProductController extends AbstractController
```

```
{
```

```
  #[Route('/products')]
```

```
  public function list(LoggerInterface $logger): Response
```

```
  {
```

```
    $logger->info('Look, I just used a service!');
```